DevCollab: System Architecture & Presentation Notes

1. Executive Summary

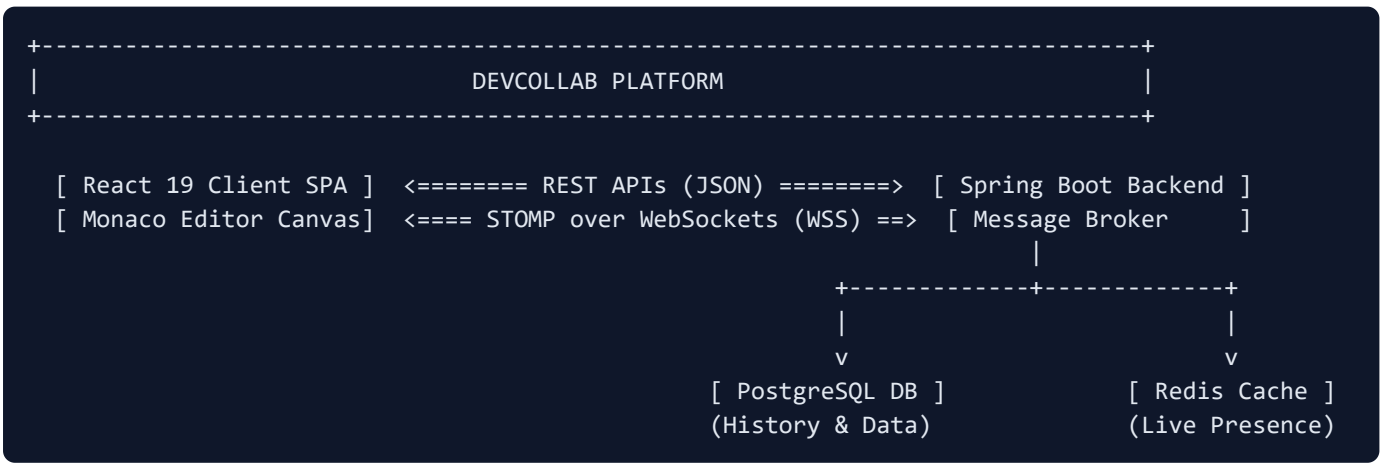
- **Product**: DevCollab is a real-time collaborative code editor in the browser.
- **Goal**: Teams, interviewers, and students write, review, and run code together.
- **Frontend**: React 19, Tailwind CSS, and Microsoft Monaco Editor.
- **Backend**: Spring Boot 4 (Java 17), PostgreSQL, and Redis.
- **Protocols**: REST APIs for data queries and STOMP over WebSockets for live sync.

2. Technology Stack

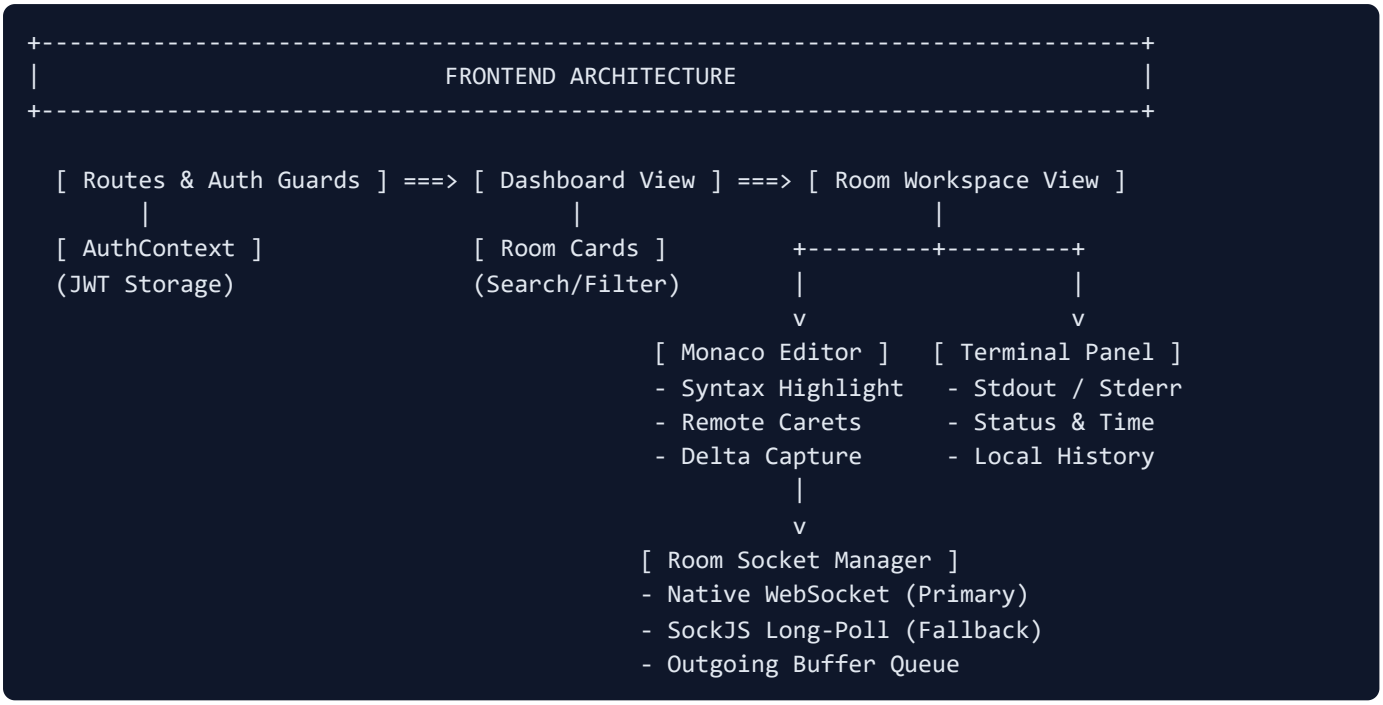
Layer	Technologies	Key Function
Frontend UI	React 19, Tailwind CSS	Responsive user interface and routing.
Code Editor	Monaco Editor (`@monaco-editor/react`)	VS Code editing experience in browser.
Real-Time Client	`@stomp/stompjs` , `sockjs-client`	Live message delivery with fallback.
Backend Core	Spring Boot 4, Spring Security	REST API, access control, and STOMP broker.
Concurrency	Operational Transformation (OT)	Conflict-free real-time document editing.
Database & Cache	PostgreSQL, Redis, Flyway	Data storage, presence cache, and migrations.
Execution Sandbox	Process Isolation (Node, Python, Java, C++, C)	Secure server-side code execution.

3. System Architecture Diagrams

3.1 Overall System Topology

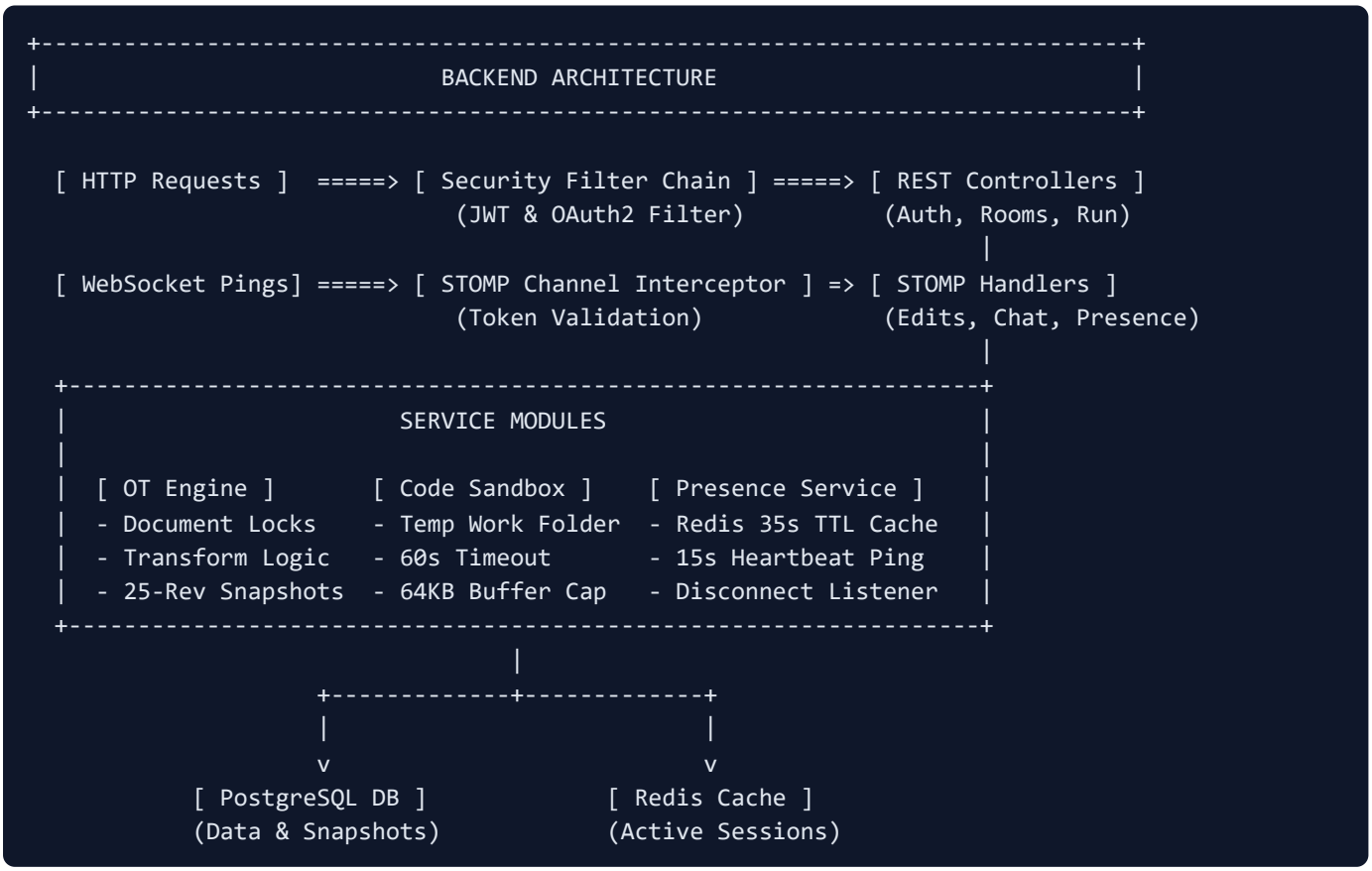


3.2 Frontend Architecture



- **Authentication Layer**: Stores JWT in local storage. Injects bearer headers on requests.
- **Workspace Canvas**: Integrates Monaco Editor. Shows colored remote carets with user names.
- **Sync Engine**: Sends keystroke deltas (`INSERT` , `DELETE`). Triggers 1.5-second full-sync safety check.
- **Output Terminal**: Shows command, execution time, exit code, and isolated local session history.

3.3 Backend Architecture



- **Security & Ingress**: Verifies JWT tokens on both HTTP requests and STOMP socket handshakes.
- **OT Engine**: Synchronizes document updates using locks. Resolves simultaneous typing conflicts.
- **Sandbox Runner**: Runs code inside temporary folders. Enforces 60-second timeouts and memory limits.
- **Presence Engine**: Uses Redis with a 35-second expiration timer to track active users.

---

4. Core Features Summary

1. User Identity & Authentication

- Supports credential login (email and password) and social OAuth2 (Google and GitHub).
- Uses stateless JWT tokens. Protects REST endpoints and WebSocket channels.

2. Room Lifecycle & Governance

- Generates unique 8-character invite codes for private access.
- Provides three permission roles:
  - **Owner**: Full control. Can change room language, kick participants, and delete room.
  - **Editor**: Can read, write, run code, and send chat messages.
  - **Viewer**: Read-only observation mode. Editor inputs are locked.

3. Real-Time Document Synchronization

- Captures typing changes as atomic deltas (`INSERT` , `DELETE` , `REPLACE`).
- Transforms concurrent edits on the server using monotonic revision numbers.
- Stores full snapshots every 25 revisions to keep history small and recovery fast.

4. Remote Carets & Presence

- Shows collaborator cursor positions with custom floating name tags.
- Removes carets after 20 seconds of inactivity or upon user exit.
- Tracks active users using Redis cache with 15-second heartbeat pings.

5. Secure Multi-Language Sandbox

- Compiles and runs 5 languages: JavaScript, Python, Java, C++, and C.
- Automatically finds Java class names to prevent compile errors.
- Enforces strict safety limits: 60-second timeout, 256MB JVM cap, and 64KB output buffer limit.

6. In-Room Group Chat

- Delivers real-time chat messages through STOMP pub-sub channels.
- Persists message history in PostgreSQL with localized UTC timestamps.

---

5. Essential Viva Questions and Answers

Q1: What is DevCollab and what core problem does it solve?

Answer: DevCollab is a browser-based collaborative code editor. It lets developers write, review, and execute code together in real time. It removes the need for screen sharing and manual code copying during technical interviews and pair programming.

Q2: Why use a decoupled architecture (React frontend + Spring Boot backend)?

Answer:

1. **Rich Client**: Monaco Editor and live cursor widgets need fast client-side DOM rendering in React.
2. **Stateless Scale**: Spring Boot acts as a stateless API and WebSocket broker that can scale horizontally.
3. **Independent Deployment**: The frontend deploys to fast static CDNs, while the backend deploys in Docker containers.

Q3: How do multiple users edit code simultaneously without conflicts?

Answer: The backend uses **Operational Transformation (OT)**:

1. When a user types, the client sends the delta operation (`INSERT` or `DELETE`) with the current revision number.
2. The server acquires a lock on the room to order incoming edits sequentially.
3. If concurrent edits occurred, the server transforms the incoming operation against previous operations so no text is lost.
4. The server increments the revision number and broadcasts the transformed edit to all room peers.

Q4: Why did you choose Operational Transformation (OT) over CRDT?

Answer:

1. **Central Server**: DevCollab uses an authoritative backend server. OT works best with a central coordinator.
2. **Low Memory**: CRDTs store tombstones for deleted characters, which increases memory use. OT keeps a clean string document.
3. **Monaco Compatibility**: Monaco's native edit API applies character ranges directly, matching OT operations.

Q5: Why use STOMP over WebSockets instead of raw WebSockets?

Answer: Raw WebSockets only provide a simple byte stream. STOMP adds standard headers, destinations, and pub-sub topics (`/topic` for broadcasts and `/app` for requests). This allows code edits, cursor coordinates, presence pings, and chat to share a single WebSocket connection cleanly.

Q6: How do you authenticate WebSocket connections?

Answer: Standard browser WebSocket APIs do not support custom HTTP headers during connection setup. The backend uses a Spring `ChannelInterceptor` that inspects the STOMP `CONNECT` frame, validates the Bearer JWT token, and binds the authenticated user to the socket session.

Q7: How are user roles (Owner, Editor, Viewer) enforced?

Answer:

1. **Database Role**: Every participant has an assigned role: `OWNER`, `EDITOR`, or `VIEWER`.
2. **Frontend Lock**: If the user is a `VIEWER`, the frontend locks the Monaco editor (`readOnly: true`).
3. **Backend Validation**: The backend rejects code modification requests from viewers.
4. **Live Role Updates**: When an owner updates a user role, the server broadcasts an event that updates permissions immediately without a page refresh.

Q8: How does the backend run untrusted user code safely?

Answer: The backend uses process-level sandbox protection:

1. **Temp Folders**: Each run creates a temporary directory that is deleted after execution.
2. **Timeout Limits**: Code has a hard 60-second execution limit. The server forcibly kills runaway processes.
3. **Memory Limits**: Java runs with memory bounds (`-Xmx256m`).
4. **Buffer Caps**: Output is capped at 64KB to avoid memory exhaustion from infinite print loops.
5. **Deadlock Prevention**: Process standard output and error output are read asynchronously in parallel threads.

Q9: How does user presence work and why is Redis used?

Answer:

1. Active clients send presence pings every 15 seconds.
2. The server saves active status in Redis with a 35-second expiration timer.
3. If a client disconnects silently, Redis expires the record automatically.
4. If Redis goes down, the system switches to an internal in-memory map without downtime.
5. When a user closes the browser tab, the server catches the disconnect event and marks them offline immediately.

Q10: What happens if a network packet drops during editing?

Answer:

1. **Reconnection Queue**: Outbound edits buffer in client memory while disconnected and flush once reconnected.
2. **Debounced Full-Sync**: 1.5 seconds after typing stops, the client verifies full content with the server. If document content differs from server truth, the editor reconciles automatically.